CAMPUSSHARE / ENGINEERING OVERVIEW

The technology behind sharing.

A campus borrowing platform for discovering items, requesting exchanges, messaging owners, confirming handovers and returns, and completing test-mode payments.

Layer	Technology	Role in this project
Frontend	React 19 + React Router 7	Reusable UI and client-side routing across marketplace, account, exchange and admin pages.
Build tooling	Vite 7 + npm	Development server, production bundles and package lockfiles.
Visual system	CSS + Tailwind CSS 4	Shared blue-and-white theme, responsive layouts, states and utility styling.
Typography and icons	DM Sans / DM Serif Display; Lucide React	Locally bundled readable fonts and consistent interface icons.
Motion and charts	Motion; Recharts	Entrance and layout motion, interactive feedback and dashboard charts. Native browser scrolling is used.
Client networking	Axios	Browser requests to the Express API.
Backend	Node.js + Express 5	HTTP API, validation, business logic, route protection and static frontend hosting.
Database	MySQL 8 + mysql2	Persistent relational data, SQL access, schema setup and demo seed data.
Authentication	bcrypt; JSON Web Tokens; cookies	Password hashing, token-based sessions and protected student/admin routes.
Security middleware	Helmet, CORS, express-rate-limit	Security headers, allowed-origin controls and request throttling.
Uploads	Multer + Cloudinary	Upload handling and optional configured cloud image storage.
Payments	Razorpay hosted checkout + server verification	Test-mode orders and signature verification; animated demo fallback when test keys are absent.

Layer	Technology	Role in this project
Testing and deployment	Node test runner; Docker / Compose	API integration tests, production build checks and container configuration.

How the parts connect

React browser UI → Express API → MySQL database. The API separately contacts Razorpay for payment verification and Cloudinary for configured uploads. Provider secrets stay on the server.

Repository map

client/ - active React application, styles and frontend assets.
server/src/ - API routes, controllers, middleware and services.
server/database/ - schema and development data setup.
server/test/ - integration tests.
public/404/ - preserved animated 404 experience.
scripts/start-full.mjs - unified local start.

Local development

Use a current Node.js release compatible with Vite 7 (Node 22.12+ recommended), npm and MySQL 8. Copy server/.env.example to server/.env and enter local values. Run npm ci, npm --prefix client ci, and npm --prefix server ci. Then run npm --prefix server run db:setup. Build with npm run build and start with npm start. Open http://localhost:3000. See README.md for separate frontend/API development.

Status and important boundaries

The production build and eight integration tests passed in the local environment. Desktop/mobile page-load checks covered 24 routes at each viewport. These checks do not replace production security, load or accessibility testing.

Payments are test-only: this build rejects live Razorpay keys. Real-money release requires reviewed live-mode support, webhook reconciliation and production configuration. Demo accounts and catalogue images must be replaced before public launch.

GSAP, Lenis and Radix packages are installed; package presence does not mean every page uses them. Lenis smooth scrolling is deliberately not enabled. The active app is React/Express/MySQL; older prototype files are not its runtime.

Sources: client/package.json, server/package.json, active source files, README.md and DESIGN-HANDOFF.md. Package families above describe this repository, not a claim about the latest available releases.